

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet omogenee sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y) \equiv 0, \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano.

Discretizzazione alle differenze finite

Si introduce una griglia cartesiana uniforme di $(N+2)^2$ punti (x_i, y_j) con $i, j = 0, \dots, N+1$ e passo $h = 1/(N+1)$, dove $x_i = i \cdot h$ e $y_j = j \cdot h$. I punti di bordo hanno temperatura fissata a zero e non contribuiscono come incognite. Le N^2 incognite del problema sono i valori $u_{i,j} \approx u(x_i, y_j)$ nei punti *interni* ($1 \leq i, j \leq N$).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (2)$$

Se un vicino cade sul bordo, il suo valore (zero) viene incorporato nel termine noto senza aggiungere incognite. L'insieme delle equazioni (2) costituisce il sistema lineare $Au = b$, dove $A \in \mathbb{R}^{N^2 \times N^2}$ è sparsa (al più 5 elementi non nulli per riga), simmetrica e definita negativa.

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si può calcolare la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Tuttavia, durante la fattorizzazione si generano elementi non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe, la larghezza di banda di A è $\Theta(N)$ e il fill-in di L è $O(N^3)$. La tecnica di *nested dissection* riduce il fill-in a $O(N^2 \log N)$, con un risparmio sostanziale per valori grandi di N .

Grafo di adiacenza e nested dissection

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare, due nodi sono collegati se e solo se sono vicini orizzontalmente o verticalmente:

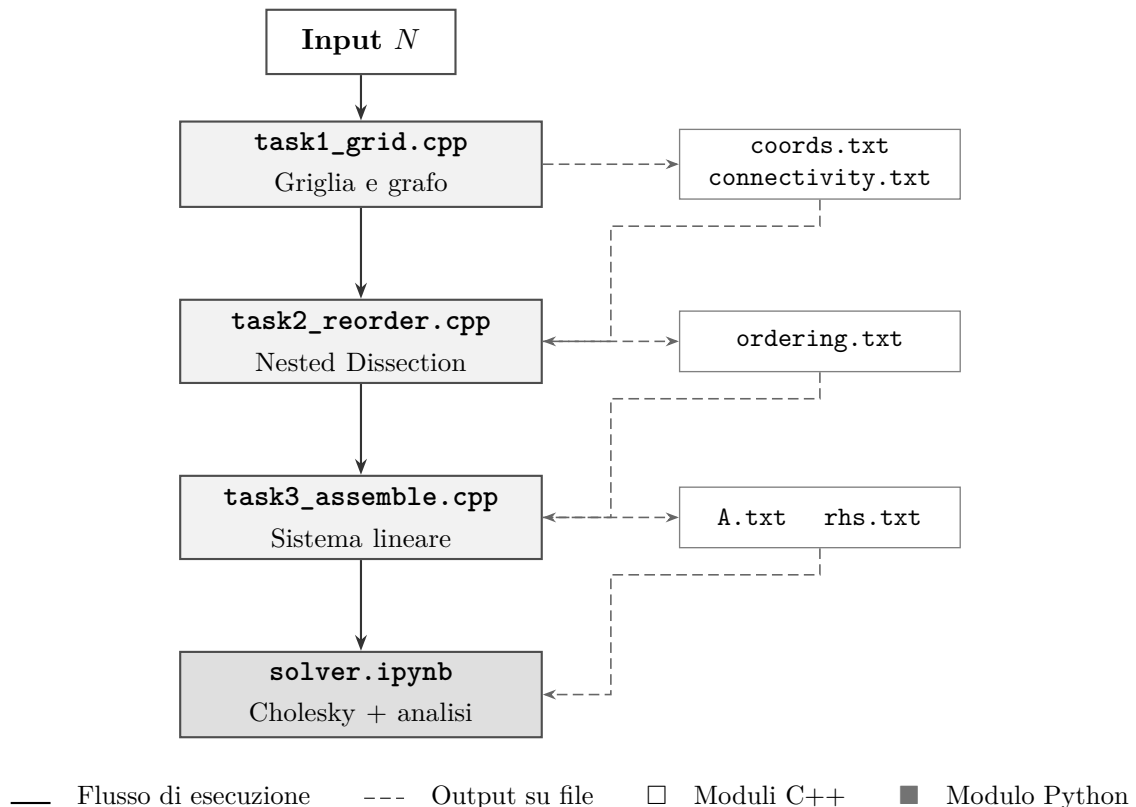
$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore geometrico V_S di cardinalità ridotta che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in. Per la griglia regolare il separatore ottimale è la colonna mediana: scelto $\hat{x}$ mediano, si pone $V_1 = \{u_{i,j} \mid x_i < \hat{x}\}$, $V_2 = \{u_{i,j} \mid x_i > \hat{x}\}$, $V_S = \{u_{i,j} \mid x_i = \hat{x}\}$. Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y .

2 Architettura del progetto

Il progetto è strutturato in quattro moduli. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python li legge come punto di ingresso per la fase numerica finale.

Il diagramma seguente illustra il flusso di elaborazione dall'input N alla soluzione e all'analisi comparativa:



Ogni modulo ha un ruolo preciso all'interno della pipeline:

task1_grid.cpp

C++

Riceve N ; genera i N^2 nodi interni con indici e coordinate (`coords.txt`) e la lista degli archi del grafo di adiacenza (`connectivity.txt`).

task2_reorder.cpp

C++

Legge il grafo e calcola la permutazione nested dissection ricorsiva: ad ogni livello individua il separatore geometrico mediano (alternando asse x e y) e lo posiziona in coda. Salva la corrispondenza in `ordering.txt`.

task3_assemble.cpp

C++

Assembla la matrice sparsa A (stencil a 5 punti: $-4\kappa/h^2$ in diagonale, $+\kappa/h^2$ fuori diagonale) e il termine noto b . Supporta ordinamento naturale e nested dissection. Produce `A.txt` e `rhs.txt`.

solver.ipynb

Python

Carica `A.txt` e `rhs.txt` in formato CSC; fattorizza $-A = LL^T$ con `sksparse.cholmod`; risolve con `spsolve_triangular`. Confronta tempi per $N = 32 \dots 1024$; produce spy plot e grafici della soluzione $u(x, y)$.

Le strutture dati fondamentali sono: liste di adiacenza (`std::vector<std::vector<int>>`) per il grafo nei moduli C++, e matrici sparse in formato CSC (`scipy.sparse.csc_matrix`) nel modulo Python.

3 Specifica dei moduli

I quattro moduli vengono di seguito descritti con livello di dettaglio crescente: per ciascuno si indicano input, output, funzionalità richieste, strutture dati e complessità attesa.

Modulo 1 — **task1_grid.cpp**: generazione della griglia e del grafo

Input. Intero $N \geq 1$ passato da riga di comando, che determina la discretizzazione uniforme del dominio $[0, 1]^2$ in una griglia $(N + 2) \times (N + 2)$.

Output. Due file di testo:

- `coords.txt` — una riga per ogni nodo interno nella forma
 $\mathbf{n} \quad \mathbf{i} \quad \mathbf{j} \quad \mathbf{x} \quad \mathbf{y}$, con $\mathbf{n} \in \{0, \dots, N^2 - 1\}$ (identificatore), indici di griglia $\mathbf{i}, \mathbf{j} \in \{1, \dots, N\}$ e coordinate $x_i = i/(N + 1)$, $y_j = j/(N + 1)$ (utili nel Modulo 2 per calcolare il separatore geometrico senza ridedurli);
- `connectivity.txt` — una riga per ogni arco $\mathbf{u} \rightarrow \mathbf{v}$ del grafo di adiacenza, con $\mathbf{u} < \mathbf{v}$, elencando tutte le coppie di nodi interni collegati orizzontalmente o verticalmente.

Funzionalità richieste.

- Generazione dei N^2 nodi interni con coordinate e identificatore numerico progressivo basato sull'ordinamento naturale per righe: $\text{id}(i, j) = (i - 1) \cdot N + (j - 1)$ per $i, j \in \{1, \dots, N\}$.
- Costruzione degli archi: per ogni nodo (i, j) si aggiungono fino a quattro archi verso $(i \pm 1, j)$ e $(i, j \pm 1)$, escludendo i punti di bordo.

- Scrittura dei file `coords.txt` e `connectivity.txt`.
- Misurazione dei tempi di esecuzione per la generazione dei nodi e del grafo tramite la libreria standard `<chrono>`.

Strutture dati.

- Array bidimensionale (o mappa) `id[i][j]` per la conversione da indici di griglia (i, j) a identificatore di nodo.
- Lista di adiacenza `std::vector<std::vector<int>>` di dimensione N^2 per memorizzare i vicini di ciascun nodo (si raccomanda l'uso di `std::vector::reserve()` per pre-allocare memoria ed evitare riallocazioni, specialmente per N grandi).

Complessità attesa. Generazione e scrittura in tempo $O(N^2)$: ogni nodo possiede al più quattro vicini e il grafo ha al più $2N(N - 1)$ archi.

Dipendenze. Nessuna dipendenza da altri moduli; produce i file letti da `task2_reorder.cpp` e `task3_assemble.cpp`.

Modulo 2 — `task2_reorder.cpp`: nested dissection

Input.

- `coords.txt` e `connectivity.txt` prodotti dal Modulo 1;
- intero N (da riga di comando o dedotto dal file).

Output. File `ordering.txt` con N^2 righe, ciascuna della forma `new_id old_id`, dove `new_id` $\in \{0, \dots, N^2 - 1\}$ è la posizione assegnata dalla nested dissection all'nodo di identificatore originale `old_id`.

Funzionalità richieste.

- Lettura del grafo di adiacenza con coordinate geometriche.
- Implementazione iterativa della nested dissection (basata su uno stack esplicito in memoria heap per prevenire stack overflow su griglie di grandi dimensioni): dato un sottografo indotto da un insieme $S \subseteq V$ con estensione geometrica $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$:
 1. scegliere l'asse di taglio (prima x , poi y in alternanza);
 2. individuare il separatore $V_S \subset S$ come l'insieme dei nodi con coordinata sull'asse di taglio pari alla mediana;
 3. processare in modo iterativo prima $V_1 = S \setminus (V_2 \cup V_S)$ (sottodominio “inferiore”), poi V_2 (sottodominio “superiore”), aggiungendo infine V_S (separatore) all'ordinamento;
- Misurazione del tempo di esecuzione dell'algoritmo tramite `<chrono>`.
- Scrittura della permutazione risultante in `ordering.txt`.

Strutture dati.

- Lista di adiacenza `std::vector<std::vector<int>>` con coordinate associate.
- Struttura dati `std::stack` per gestire in modo iterativo i sottoproblemi da elaborare (emulando lo stack di chiamate).
- Array `perm[]` di dimensione N^2 che accumula la permutazione finale.

Complessità attesa. La nested dissection su griglia $N \times N$ richiede $O(N^2 \log N)$ operazioni; la scrittura del file è $O(N^2)$.

Dipendenze. Legge i file prodotti dal Modulo 1; produce `ordering.txt` usato dal Modulo 3.

Modulo 3 — `task3_assemble.cpp`: assemblaggio del sistema lineare

Input.

- Intero N (da riga di comando).
- Flag opzionale `-reorder` che indica se usare l'ordinamento da `ordering.txt` anziché quello naturale.
- File `coords.txt` (per le coordinate dei nodi e il termine sorgente f) e, se richiesto, `ordering.txt`.

Output.

- `A.txt` — elenco delle entrate non nulle della matrice: una riga per entrata nella forma $i \quad j$ `A[i, j]` rispetto all'ordinamento scelto;
- `rhs.txt` — vettore dei termini noti: una riga per entrata contenente il valore b_k , con le righe nello stesso ordine di `A.txt`.

Funzionalità richieste.

- Calcolo del passo $h = 1/(N + 1)$ e del coefficiente $\kappa = 0.01$.
- Per ogni nodo interno k (nel nuovo ordinamento): inserimento dell'entrata diagonale $A[k, k] = -4\kappa/h^2$ e delle entrate fuori diagonale $A[k, k'] = +\kappa/h^2$ per ogni vicino k' ; se un vicino k' cade sul bordo, l'entrata corrispondente sparirà dalla matrice e il suo valore ($u_0 = 0$) andrà portato al secondo membro b_k . Anche se l'effetto numerico è nullo, la logica deve essere robusta per condizioni non omogenee.
- Valutazione del termine sorgente: $b_k = -f(x_{i(k)}, y_{j(k)})$ con $f(x, y) = e^{-10(x^2+y^2)}$.
- Applicazione della permutazione (se richiesta) per tradurre gli indici dal sistema naturale al sistema riordinato.

Invariante. La matrice prodotta è simmetrica: ogni coppia (i, j) con $i \neq j$ compare due volte in `A.txt` con lo stesso valore.

Strutture dati.

- Array `perm[]` per la permutazione; `inv_perm[]` per l'inversa (`inv_perm[old_id] = new_id`), fondamentale per mappare l'ordinamento in tempo $O(N^2)$.
- Tripli (i, j, val) accumulati in un `std::vector` prima della scrittura (anche qui, usare `reserve()` conoscendo la numerosità attesa).

Complessità attesa. Assemblaggio in $O(N^2)$ (al più 5 entrate per nodo); scrittura in $O(\text{nnz}) = O(N^2)$.

Dipendenze. Legge `coords.txt` e opzionalmente `ordering.txt`; produce i file letti dal Modulo 4.

Modulo 4 — `solver.ipynb`: soluzione e analisi

Input.

- `A.txt` e `rhs.txt` in formato sparso (i, j, val) / vettore.
- Parametro N (o serie di valori $N \in \{32, 64, 128, 256, 512, 1024\}$).

Output.

- Soluzione $u \in \mathbb{R}^{N^2}$ del sistema $Au = b$ (riordinata nella griglia originale per la visualizzazione).
- Spy plot della struttura di A prima e dopo il riordinamento.
- Grafico della soluzione $u(x, y)$ come superficie o heatmap 2D.
- Tabella/grafico comparativo di tempi di fattorizzazione e numero di elementi non nulli di L per le due strategie di ordinamento.

Funzionalità richieste.

- Lettura dei file e costruzione di `scipy.sparse.csc_matrix` (o `coo_matrix`), che gestiscono automaticamente la somma delle entrate duplicate dovute alla simmetria del file `A.txt`.
- Fattorizzazione $-A = LL^T$ tramite `sksparse.cholmod.cholesky` con `order="natural"` (l'ordinamento è già codificato in A).
- Soluzione del sistema lineare tramite `scipy.sparse.linalg.spsolve_triangular`.
- Misurazione del tempo di esecuzione (con `time.perf_counter`) per ciascun valore di N e per entrambe le strategie di ordinamento.
- Confronto del fill-in: conteggio di `nnz(L)` per ordinamento naturale e nested dissection.

Strutture dati.

- `scipy.sparse.csc_matrix` per A (e $-A$ per la fattorizzazione).
- Array `numpy.ndarray` per il vettore b e la soluzione u .
- Oggetto `Factor` di `CHOLMOD` per la fattorizzazione.

Complessità attesa. Con ordinamento naturale: fill-in $O(N^3)$ e risoluzione $O(N^3)$. Con nested dissection: fill-in $O(N^2 \log N)$ e risoluzione $O(N^2 \log N)$.

Dipendenze. Legge `A.txt` e `rhs.txt` prodotti dal Modulo 3; non genera file ma produce output grafico nel notebook.

4 Analisi sperimentale attesa

L'obiettivo dell'analisi sperimentale è quantificare il vantaggio computazionale della nested dissection rispetto all'ordinamento naturale. Si prevedono le seguenti misurazioni per $N \in \{32, 64, 128, 256, 512, 1024\}$:

Grandezza misurata	Ordinamento naturale	Nested dissection
<code>nnz(A)</code>	$O(N^2)$	$O(N^2)$
<code>nnz(L)</code> (fill-in)	$O(N^3)$	$O(N^2 \log N)$
Tempo di fattorizzazione	$O(N^3)$	$O(N^2 \log N)$ (atteso)
Tempo di risoluzione	$O(N^3)$	$O(N^2 \log N)$ (atteso)

Il notebook produrrà:

1. Spy plot di A e di L (fattore di Cholesky) per entrambi gli ordinamenti, per evidenziare visivamente la differenza di fill-in.

2. Heatmap 2D o superficie 3D della soluzione $u(x, y)$ per almeno un valore di N .
3. Grafico log-log di $\text{nnz}(\mathbf{L})$ in funzione di N : le due curve dovrebbero separare nettamente attorno a $N \geq 64$. Verrà esplicitamente calcolato anche il rapporto $\text{nnz}(L_{\text{natural}})/\text{nnz}(L_{\text{ND}})$ (fill-in relativo) per evidenziare il risparmio di memoria.
4. Grafico dei tempi di fattorizzazione in funzione di N , con indicazione della pendenza empirica per stimare l'esponente di crescita.